

Divide-and-conquer Approach

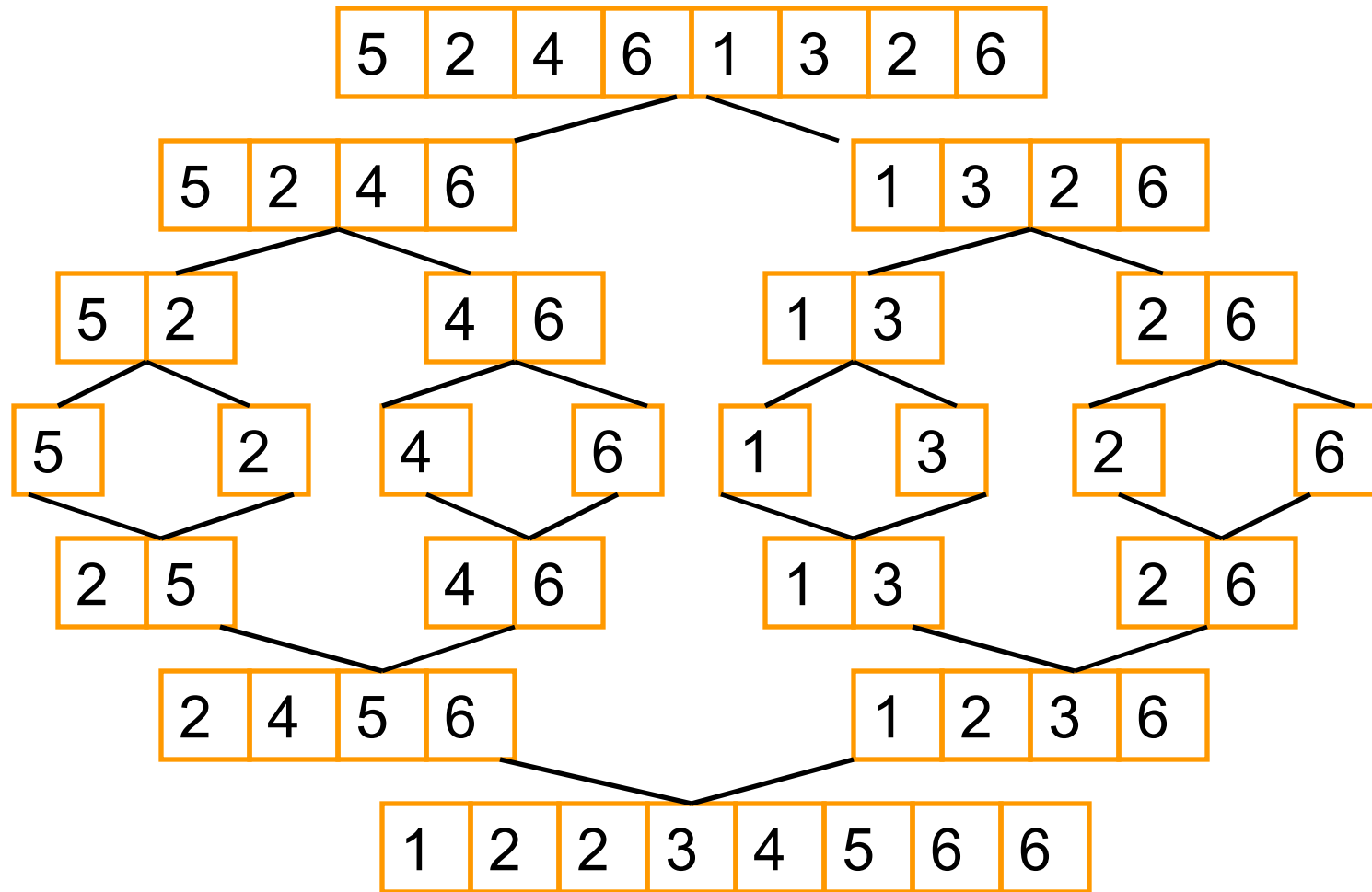
Divide and Conquer Paradigm

- An important general technique for designing algorithms:
 - divide problem into subproblems
 - recursively solve subproblems
 - combine solutions to subproblems to get solution to original problem
- Use recurrences to analyze the running time of such algorithms

Example: Mergesort

- DIVIDE the input sequence in half
- RECURSIVELY sort the two halves
 - basis of the recursion is sequence with 1 key
- COMBINE the two sorted subsequences by merging them

Mergesort Example



Multiplication of Two Complex Numbers

Consider two complex numbers

$$Z_1 = a + i \cdot b$$

$$Z_2 = c + i \cdot d$$

- $Z_1 \cdot Z_2 =$

$$(a + bi)(c + di) = ac - bd + (bc + ad)i$$

$$bc + ad = (a + b)(c + d) - ac - bd.$$

Suppose x and y are two n -bit integers, and assume for convenience that n is a power of 2.

As a first step toward multiplying x and y , split each of them into their left and right halves, which are $n/2$ bits long:

$$x = \boxed{x_L} \boxed{x_R} = 2^{n/2}x_L + x_R$$

$$y = \boxed{y_L} \boxed{y_R} = 2^{n/2}y_L + y_R.$$

$$xy = (2^{n/2}x_L + x_R)(2^{n/2}y_L + y_R) = 2^n x_L y_L + 2^{n/2}(x_L y_R + x_R y_L) + x_R y_R.$$

- We will compute xy via the expression on the right. The additions take linear time, as do the multiplications by powers of 2 (which are merely left-shifts).
- The significant operations are the four $n/2$ -bit multiplications, $x_L y_L$, $x_L y_R$, $x_R y_L$, $x_R y_R$; these we can handle by four recursive calls.
- Thus our method for multiplying n -bit numbers starts by making recursive calls to multiply these four pairs of $n/2$ -bit numbers (four subproblems of half the size), and then evaluates the preceding expression in $O(n)$ time.

- Writing $T(n)$ for the overall running time on n -bit inputs, we get the *recurrence relation*

$$T(n) = 4T(n/2) + O(n).$$

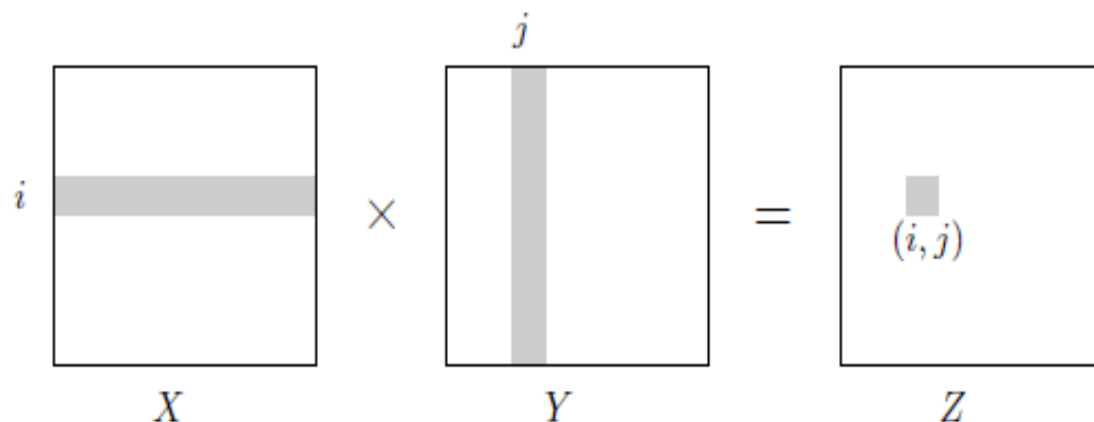
- Although the expression for xy seems to demand four $n/2$ -bit multiplications, as before just three will do: $x_L y_L$, $x_R y_R$, and $(x_L + x_R)(y_L + y_R)$, since $x_L y_R + x_R y_L = (x_L + x_R)(y_L + y_R) - x_L y_L - x_R y_R$.
- The resulting algorithm has an improved running time of $T(n) = 3T(n/2) + O(n)$.

Matrix multiplication

The product of two $n \times n$ matrices X and Y is a third $n \times n$ matrix $Z = XY$, with (i, j) th entry

$$Z_{ij} = \sum_{k=1}^n X_{ik} Y_{kj}.$$

To make it more visual, Z_{ij} is the dot product of the i th row of X with the j th column of Y :



Matrix multiplication is particularly easy to break into subproblems, because it can be performed *blockwise*. To see what this means, carve X into four $n/2 \times n/2$ blocks, and also Y :

$$X = \begin{bmatrix} A & B \\ C & D \end{bmatrix}, \quad Y = \begin{bmatrix} E & F \\ G & H \end{bmatrix}.$$

Then their product can be expressed in terms of these blocks and is exactly as if the blocks were single elements (Exercise 2.11).

$$XY = \begin{bmatrix} A & B \\ C & D \end{bmatrix} \begin{bmatrix} E & F \\ G & H \end{bmatrix} = \begin{bmatrix} AE + BG & AF + BH \\ CE + DG & CF + DH \end{bmatrix}$$

We now have a divide-and-conquer strategy: to compute the size- n product XY , recursively compute eight size- $n/2$ products $AE, BG, AF, BH, CE, DG, CF, DH$, and then do a few $O(n^2)$ -time additions. The total running time is described by the recurrence relation

$$T(n) = 8T(n/2) + O(n^2).$$

- This comes out to an unimpressive $O(n^3)$, the same as for the default algorithm.
- But the efficiency *can* be further improved, and as with integer multiplication, the key is some clever algebra. It turns out XY can be computed from just *seven* $n/2 \times n/2$ subproblems, via a decomposition so tricky and intricate that one wonders how Strassen was ever able to discover it!

Strassen Matrix Multiplication

$$XY = \begin{bmatrix} P_5 + P_4 - P_2 + P_6 & P_1 + P_2 \\ P_3 + P_4 & P_1 + P_5 - P_3 - P_7 \end{bmatrix}$$

where

$$\begin{array}{ll} P_1 &= A(F - H) & P_5 &= (A + D)(E + H) \\ P_2 &= (A + B)H & P_6 &= (B - D)(G + H) \\ P_3 &= (C + D)E & P_7 &= (A - C)(E + F) \\ P_4 &= D(G - E) \end{array}$$

The new running time is

$$T(n) = 7T(n/2) + O(n^2),$$

which by the master theorem works out to $O(n^{\log_2 7}) \approx O(n^{2.81})$.

Merge sort

- Merge sort is a classic, efficient, and widely-used sorting algorithm based on the **divide-and-conquer** paradigm. Here's an overview of its basic concepts:
- **Divide and Conquer**
- **Divide:** Split the array into two halves.
- **Conquer:** Recursively sort each half using the merge sort algorithm.
- **Combine:** Merge the two sorted halves to produce the sorted array.

- Splitting the Array:**

- The array is divided into smaller subarrays until each subarray contains
- only one element (which is inherently sorted).

- Merging the Subarrays:**

- Pairs of subarrays are merged into sorted larger arrays.
- This merging process continues until you end up with one sorted array.

Pseudocode

```
function mergeSort(arr):  
    { int mid;  
    if length of arr <= 1: return  
    mid = length of arr / 2  
    leftHalf = mergeSort(arr[0:mid]) //  
    rightHalf = mergeSort(arr[mid+1:end])  
    return merge(leftHalf, rightHalf)  
}
```

- function merge(left, right):
 { result = []
 i = 0 j = 0
 while i < length_of_left and j < length of right:
 if left[i] <= right[j]:
 append left[i] to result;
 i = i + 1
 else:
 append right[j] to result
 j = j + 1
 while i < length_of_left:
 append left[i] to result
 i = i + 1
 while j < length_of_right:
 append right[j] to result
 j = j + 1
 return result
 }

Quick Sort

- Quick Sort is a highly efficient sorting algorithm based on the **divide-and-conquer** paradigm.
- Unlike merge sort, which divides the array into equal halves, quick sort selects a "pivot" element and partitions the array around the pivot, so that elements less than the pivot are on one side, and elements greater than the pivot are on the other side.
- The process is then recursively applied to the partitions.

- `function quickSort(arr, low, high):`
 - `if low < high: pivotIndex = partition(arr, low, high)`
 - `quickSort(arr, low, pivotIndex - 1) // Recursively sort the left part`
 - `quickSort(arr, pivotIndex + 1, high) // Recursively sort the right part`

- function partition(arr, low, high):
 pivot = arr[high] // Choose the rightmost element as the pivot
 i = low - 1 // Pointer for the greater element
 for j = low to high - 1:
 if arr[j] <= pivot:
 i = i + 1 // Increment the smaller element index
 swap arr[i] with arr[j]
 // Swap elements if current element is smaller than or equal to pivot
 swap arr[i + 1] with arr[high]
 // Swap the pivot element with the element at i+1
 return i + 1 // Return the pivot index

- **Pivot Selection:** The choice of pivot can affect the efficiency. Common strategies include selecting the first, last, middle, or a random element.
- **Partitioning:** Partitioning rearranges the array so that elements smaller than the pivot come before it and elements larger come after it.
- **Recursion:** After partitioning, quick sort is recursively applied to the subarrays formed by the partition.

Binary Heap

- A heap is a complete binary tree, meaning all levels are fully filled except possibly the last level, which is filled from left to right. Each node has at most two children.
- In a **max-heap**, each parent node is greater than or equal to its children, ensuring that the maximum element is always at the root.
- In a **min-heap**, each parent node is less than or equal to its children, ensuring the minimum element is at the root.

- The root of the heap (the first element in the array representation) provides constant-time access to the maximum element in a max-heap or the minimum element in a min-heap.
- In max heap, going from root print elements in descending order
- Similarly in min heap, going from root print elements in ascending order of the values.

- Key operations include **insertion** (adding a new element while maintaining the heap property), **deletion** (typically removing the root element), and **heapify** (ensuring a subtree satisfies the heap property).
- These operations have logarithmic time complexity, $O(\log n)$ due to the tree's height

Heapify Pseudocode

HEAPIFY(A, i, n):

{ left = 2 * i

right = 2 * i + 1

largest = i

if (left <= n and A[left] > A[largest])

largest = left

if (right <= n and A[right] > A[largest])

largest = right

if (largest != i)

{swap(A[i], A[largest])

HEAPIFY(A, largest, n)

}

Insert into Max Heap

INSERT(A, key, n):

{

$n = n + 1$

$A[n] = \text{key}$

$i = n$

 while ($i > 1$ and $A[\text{PARENT}(i)] < A[i]$):

 swap($A[i]$, $A[\text{PARENT}(i)]$)

$i = \text{PARENT}(i)$

}

PARENT(i): return floor($i / 2$)

Delete from Max Heap

```
DELETE_MAX(A, n):  
{  
    if (n < 1):  
        error 'Heap underflow'  
  
        max = A[1];  
        A[1] = A[n];  
        n = n - 1;  
        HEAPIFY(A, 1, n);  
    return max;  
}
```